

OVERTAKE

AI-Powered Formula 1 Race Strategy & Performance Intelligence

Project Planning & Technical Design Document

Overtake is a challenging ML/AI portfolio project designed to model how a Formula 1 race engineer could use telemetry, tyre behavior, lap-time forecasting, weather and race-state information to make better strategic decisions.

Core idea: turn raw F1 data into predictions, simulate thousands of possible race futures, and recommend the strategy with the highest expected performance.

1. What Are We Building?

Overtake will be a complete F1 analytics and decision-support system rather than a single ML model. The system will ingest historical Formula 1 telemetry and race information, learn how lap pace and tyres behave, simulate possible race strategies, and produce an explainable recommendation.

The long-term vision is an **AI race engineer**: a system that can answer questions such as “Should we pit now?”, “Which tyre should we use?”, “How long can this tyre last?”, and “What is our probability of finishing on the podium?”

The first version will not depend on live race access. Instead, historical races will be replayed lap-by-lap as if they were happening in real time. This makes the system testable and allows us to compare every AI decision with what actually happened.

2. Main Objectives

- Build a clean, reusable F1 telemetry and race-data pipeline.
- Predict tyre degradation and future lap pace.
- Forecast upcoming lap/sector performance.
- Simulate alternative pit-stop and tyre strategies.
- Use Monte Carlo simulation to quantify uncertainty and race outcomes.
- Identify where a driver is gaining or losing time around a circuit.
- Build an interactive dashboard showing predictions and recommendations.
- Eventually support real-time data ingestion when a suitable live source is available.

3. Data Source

The primary starting point will be **OpenF1**, which provides publicly accessible historical Formula 1 data including session information, laps, car telemetry, positions, weather, tyres, pit stops and race-control information. Historical data can be used without building a live-data dependency.

For development, we will download and cache the required data locally so that experiments are reproducible and the ML pipeline does not depend on an external API being available every time.

Important distinction: historical/replay mode is the core project. Live race data is an optional later feature and may require a paid data-access tier or another permitted source.

4. Overall Architecture

The project will be organized as a pipeline:

F1 Data → Data Cleaning → Feature Engineering → ML Models → Race Simulator → Strategy Optimizer → Recommendation → Dashboard

Three main prediction components feed the strategy engine:

Component	Purpose	Initial approach
Tyre model	Estimate pace loss as tyres age	XGBoost / gradient boosting
Lap-time model	Predict upcoming lap/sector performance	XGBoost → LSTM/Transformer
Race simulator	Generate possible future race states	Monte Carlo simulation
Strategy optimizer	Choose pit/tyre decisions	Search + optimization; RL later
Performance analysis	Find time loss by sector/corner	Telemetry comparison + ML

5. Phase 1 — Data Engineering

First, we build the data foundation. We select several races across different circuits and conditions, retrieve telemetry and race-state information, normalize timestamps/laps, handle missing values, and store the processed data in efficient local formats such as Parquet, with DuckDB/PostgreSQL for querying where useful.

Example features: driver, team, circuit, lap number, sector, speed, throttle, brake, gear, RPM, tyre compound, tyre age, lap time, sector time, gap, position, pit status, track temperature, air temperature and race-control events.

A key rule is **no data leakage**. When simulating lap 20, the system must only see information that would have been available at or before lap 20.

6. Phase 2 — Tyre Degradation Model

The first major ML model predicts how tyre performance changes as a stint progresses. The target is typically lap time or pace loss, while inputs include compound, tyre age, circuit, track temperature, recent lap pace and other available race-state features.

We will start with interpretable and strong tabular models such as XGBoost rather than immediately using deep learning. The model can produce a degradation curve for a stint.

Example output:

```
Lap 20 → 91.8 s
Lap 21 → 92.1 s
Lap 22 → 92.5 s
Lap 23 → 93.0 s
Lap 24 → 93.7 s
```

From this curve, the system can estimate when staying on a tyre becomes strategically expensive and compare that cost with the time lost by pitting.

7. Phase 3 — Lap-Time & Sector Prediction

The next model predicts the driver's near-future pace. Instead of predicting only the final race result, it predicts the next lap or several upcoming laps.

Potential inputs include the driver, circuit, tyre compound and age, previous lap/sector times, track conditions, gaps, traffic indicators and race state.

We will benchmark multiple models—such as linear baselines, random forests, gradient boosting and temporal neural networks—to determine whether sequence models genuinely improve performance.

This creates a defensible ML experiment: the project will report MAE/RMSE, compare models, analyze errors, and explain when predictions become less reliable.

8. Phase 4 — Historical Race Replay Engine

This is the key mechanism that makes the project feel real-time without requiring a live F1 feed.

Choose a completed race. The simulator starts at lap 1. At each lap it exposes only the data available up to that point, runs the prediction models, evaluates possible strategies, records the recommendation, then advances to the next lap.

Replay loop:

```
Race state → update features → predict tyre/lap pace → simulate strategies → rank strategies
→ record recommendation → advance one lap
```

At the end, we can compare what Overtake recommended at each decision point against the actual race and calculate whether its choices would have improved the outcome.

9. Phase 5 — Race Strategy Simulation

At a given lap, Overtake generates candidate strategies. For example:

Strategy A: stay out → pit on lap 30 → Medium
Strategy B: pit now → Hard
Strategy C: stay out → pit on lap 37 → Soft

For each strategy, the system uses the tyre and lap-time models to estimate the remaining race. It accounts for expected pace, tyre degradation, pit-lane loss, traffic/gaps and relevant race events.

10. Phase 6 — Monte Carlo Race Simulation

A single deterministic prediction is not enough because races are uncertain. We will therefore run many simulations for each candidate strategy, sampling plausible outcomes around model predictions.

For example, 10,000 simulated futures can produce:

- Probability of finishing P1/P2/P3
- Expected finishing position
- Expected total race time
- Probability of losing positions
- Strategy risk/variance
- Expected time gain or loss versus alternatives

This converts the project from simple prediction into **decision-making under uncertainty**.

11. Phase 7 — Strategy Optimizer

The optimizer chooses the strategy with the best expected outcome according to an objective function. Initially this can use exhaustive search over feasible pit windows and tyre sequences combined with Monte Carlo simulation.

Once the baseline is robust, reinforcement learning can be explored as an advanced extension. The race becomes an environment, the race state becomes the observation, pit/tyre decisions become actions, and finishing performance becomes part of the reward.

RL is deliberately a later phase. A strong simulator and optimization baseline are more important than adding reinforcement learning simply for the label.

12. Phase 8 — Driver & Corner Performance

Telemetry will also be used to explain where lap time is being lost. A lap can be segmented into sectors and, where reliable track mapping is available, individual corners.

The system can compare speed, braking, throttle application, gear and corner-exit performance against a selected benchmark or the driver's own best reference.

Example insight: Turn 4 — lower minimum speed and later throttle application correspond to an estimated 0.21 s loss.

This module makes the dashboard useful not only for strategy but also for driver-performance analysis.

13. Dashboard

A web dashboard will present the system as a complete product rather than a collection of notebooks.

- Race view:** live-style replay, positions, gaps, tyre age and predicted pace.
- Strategy view:** current recommended pit window, tyre recommendation and alternative strategies.
- Simulation view:** finishing-position probabilities and race-time distributions.
- Tyre view:** degradation curves and predicted stint life.
- Driver view:** sector/corner losses and telemetry comparisons.
- Model view:** prediction accuracy, feature importance and uncertainty.

14. Proposed Technology Stack

- Language:** Python
- Data:** Pandas, NumPy, DuckDB, Parquet, PostgreSQL (optional)
- ML:** scikit-learn, XGBoost, PyTorch
- Simulation:** NumPy/SciPy + custom Monte Carlo engine
- Backend:** FastAPI
- Frontend:** React + TypeScript + Plotly
- Deployment:** Docker + a cloud hosting platform
- Testing:** pytest + data/model validation checks

15. Suggested Repository Structure

```
overtake/  
  data/ # raw/processed data references  
  notebooks/ # exploration and experiments  
  src/  
    ingestion/ # API/data loaders  
    preprocessing/ # cleaning + feature engineering  
    models/ # tyre + lap prediction models  
    simulation/ # race replay + Monte Carlo  
    strategy/ # optimization logic  
    evaluation/ # backtesting + metrics  
  backend/ # FastAPI service  
  frontend/ # dashboard  
  tests/  
  configs/  
  Dockerfile  
  README.md
```

16. Development Roadmap

Stage	Deliverable
1	Collect and understand historical F1 data
2	Build cleaning + feature engineering pipeline
3	Train and evaluate tyre degradation model
4	Train and benchmark lap-time forecasting models
5	Build race replay engine
6	Implement strategy generation and pit-window simulation

Stage	Deliverable
7	Add Monte Carlo race simulation
8	Backtest recommendations across multiple races
9	Add driver/corner performance analysis
10	Build dashboard and API
11	Containerize and deploy
12	Optional: live data + reinforcement learning

17. How We Will Measure Success

The project should be evaluated scientifically, not only visually.

ML metrics: MAE/RMSE for lap-time and tyre predictions; calibration and ranking quality for probabilistic outputs. Strategy metrics: expected race time, finishing-position probability, pit-window accuracy and regret versus the best hindsight strategy.

Backtesting: run the system on multiple circuits, drivers, tyre compounds and race conditions to test generalization.

A particularly strong experiment is to make decisions at many points in historical races and compare Overtake's recommendation with the strategy that would have produced the best hindsight outcome.

18. What Makes Overtake Resume-Worthy?

The strength of the project is the combination of multiple areas of engineering and AI:

- Real-world time-series and telemetry data
- Feature engineering and data pipelines
- Supervised ML and model benchmarking
- Uncertainty-aware prediction
- Monte Carlo simulation
- Optimization and decision-making
- Backtesting without data leakage
- Interactive data visualization
- Production-style API and deployment
- Optional reinforcement learning and real-time ingestion

This gives the project a strong story: **observe** → **predict** → **simulate** → **optimize** → **explain**.

19. Challenges We Should Expect

F1 data is noisy and affected by many variables. Fuel load may not be directly observable, traffic can distort lap times, safety-car periods create unusual behavior, and weather can change rapidly. Different circuits also behave very differently.

Therefore, the project should avoid claiming that it can perfectly predict races. The goal is to build a measurable decision-support system and clearly communicate uncertainty and limitations.

20. Achievability

Yes. The complete vision is ambitious, but it can be developed incrementally. A useful MVP can be built around historical replay + tyre degradation + lap prediction + basic strategy simulation. Monte Carlo optimization, driver analysis, a production dashboard and live ingestion can then be layered on top.

The key is to avoid starting with the hardest component. Build the data pipeline first, establish reliable baselines, validate each model, and only then connect the components into the strategy engine.

21. Final Product Vision

When finished, a user should be able to select a historical F1 race, jump to any lap, and see what Overtake would have recommended at that exact moment.

Example:

LAP 26 / 57
Current tyre: Medium – age 11 laps
Predicted degradation: High
Recommended action: PIT
Tyre: Hard
Optimal pit window: Lap 26–28
Expected race-time gain: +4.2 s
Predicted podium probability: 61%
Confidence: 78%

The user can then compare that recommendation with what actually happened in the race. That replay/backtesting capability is the centerpiece of Overtake.

22. One-Line Project Definition

Overtake is an AI-driven F1 race engineer that learns tyre and lap-time behavior, simulates race futures, and optimizes pit-stop strategy using real-world telemetry and probabilistic modeling.

Suggested tagline: **“From telemetry to strategy.”**